

MeetFlow

Technical Requirements Document (TRD)

Version 1.0 | AI-Powered Meeting Platform for Colleges

1. Document Purpose

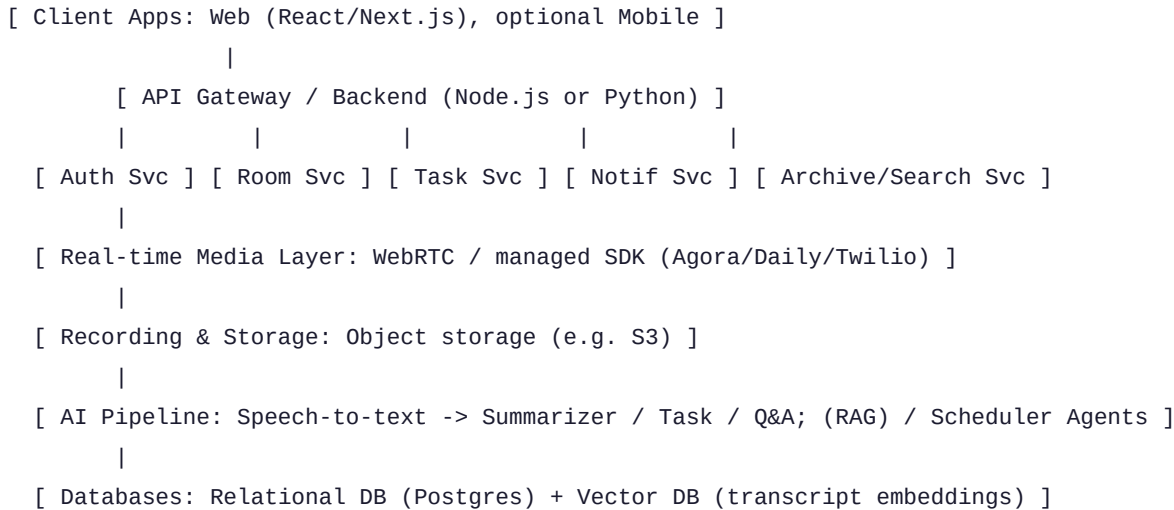
This Technical Requirements Document (TRD) translates the product requirements defined in the PRD into system-level technical specifications: architecture, technology choices, module breakdown, APIs, data flow, and non-functional engineering requirements. It is the companion to the PRD (what to build) and the Backend Schema / System Architecture Document (how data is structured).

2. System Overview

MeetFlow is a web (and optionally mobile) application composed of a real-time video-conferencing layer, a standard application backend (auth, rooms, tasks, calendars), and an AI processing pipeline (transcription, summarization, task extraction, RAG-based Q&A;) that operates on recorded meeting data. A human host approves AI output before it becomes visible to other users.

3. Architecture Overview

High-level layered architecture:



4. Technology Stack

Layer	Technology
Frontend (Web)	React / Next.js

Mobile (optional)	React Native / Flutter
Backend / API	Node.js (Express/NestJS) or Python (FastAPI/Django)
Real-time video/audio	WebRTC, or managed SDK (Agora / Daily / Twilio Video)
Relational database	PostgreSQL
Vector database	Pinecone / Weaviate (transcript embeddings for Q&A Agent)
Speech-to-text	Whisper (or equivalent ASR API)
LLM layer	LLM API for summarization, RAG-based Q&A, task extraction
Authentication	OAuth (Google Sign-In) + email/password
Notifications	Push notifications, email (e.g. SendGrid), WhatsApp Business API (optional)
Object storage	Cloud storage (e.g. AWS S3) for recordings and exported PDFs

5. Module-by-Module Technical Specification

5.1 Authentication & User Management

- Google OAuth 2.0 and email/password sign-up flows.
- Stores birth-year-derived age range (Below 18 / Above 18) and language preference at signup.
- Role is NOT a fixed account attribute — it's computed per meeting (host if user created/owns the room, participant otherwise), so the same user_id can appear as host on one room and participant on another.

5.2 Room & Session Management

- Generates a unique, collision-resistant room code per meeting (e.g. short alphanumeric ID) at creation/scheduling time.
- Stores per-room settings: password (hashed if enabled), waiting-room flag, join-before-host flag, auto-record flag, default audio mode, join mode (auto-admit vs. manual).
- Supports many concurrent active rooms; each room's real-time session is isolated (separate media session/channel).
- Reschedule operation updates a room's scheduled_start/scheduled_end fields while preserving its room_code — any calendar_entry rows referencing that room_code reflect the change automatically (see Section 6, Backend Schema doc).

5.3 Real-Time Communication Layer

- WebRTC (or managed SDK) handles signaling, media routing, and screen-share tracks for all participants (host, co-host, participant roles all get publish permission for screen share).
- Active-speaker detection feeds the "no one pinned -> show active speaker" behavior; pin state is a per-client (or optionally per-room, if host-controlled) override.
- In-meeting chat, polls, reactions, and hand-raise state are transmitted via a real-time channel (e.g. WebSocket) separate from media tracks, for lower latency on text/control events.

5.4 Recording & Storage

- Recording starts only after explicit consent (host-triggered, or auto-record per room settings) and triggers an in-meeting notice to all participants.
- Recorded media is stored in object storage, keyed by room_code + meeting instance ID (a recurring room can have multiple dated instances).
- Chat logs (including polls and shared files) are persisted and linked to the same meeting instance.

5.5 AI Processing Pipeline

- Step 1 — Speech-to-text: recording is transcribed (with speaker labels where possible).
- Step 2 — Summarizer Agent: transcript -> draft summary (key points, decisions, action items), written with status = pending_review, visible only to the host's user_id.
- Step 3 — Task Agent: transcript is scanned for action-item patterns and creates task rows with a source = 'auto_detected' flag, distinct from host-assigned tasks.
- Step 4 — Indexing: transcript is chunked and embedded into the vector database, tagged with meeting_instance_id, for the Q&A; Agent's retrieval.
- Step 5 — Highlight reel generation: the pipeline selects the top N moments correlated with detected decisions/action items and clips corresponding video segments, stored alongside the meeting instance.

5.6 Q&A; Agent (RAG)

- Given a user_id and a selected meeting_instance_id (chosen by the user from their own attended-meetings list, not a typed code), retrieves relevant transcript chunks via vector similarity search scoped to that meeting_instance_id (or across all meeting_instance_ids the user attended, for multi-meeting search).
- Returns an answer with a confidence score; if below a defined threshold, the system does not answer — it creates an escalation record targeted at a host_id selected by the user (from the list of hosts of meetings the user has attended) and notifies that host.
- Host's manual answer is stored and linked to the original question so future matching queries can be auto-answered without a new LLM call, and surfaces in the host's Doubt Log aggregation.

5.7 Task Service

- Supports both host-assigned tasks (single assignee or Select All -> fan-out to all current participants) and auto-detected tasks from the Task Agent.
- Each task row tracks assignee_id, due_at (date+time), progress_percent (0-100, or a done flag), and optional attachment reference.
- A scheduled job checks for tasks past due_at with progress < 100% and creates a reminder notification targeted at the task's host_id.
- Dashboard queries group tasks by (host_id, class_or_group_label) for the class/group-organized view.

5.8 Notification & Reminder Service

- Reminder Agent only queues a notification for a user_id + meeting_instance_id pair that exists in that user's calendar_entries table (i.e., explicitly added) — it does not notify every invitee by default.
- Notification delivery via push and/or email; avoids blanket meeting-name notifications in a multi-meeting environment per the product's notification-at-scale requirement.

5.9 Archive & Search Service

- For a user_id, an Archive entry is visible without any code check whenever that user_id is present in the meeting's attendee list (host_id or participants list) for that meeting_instance.
- For a user_id NOT in the attendee list (an absentee), the system requires a one-time verification: the user submits the room_code, and on success creates an access_grant row (user_id, meeting_instance_id) — after which the entry behaves exactly like an attended entry for all future Archive/search access.
- Search supports lookup by topic/title (full-text search) in addition to date-based browsing.

5.10 Faculty Availability & Scheduler Agent

- Stores a simple status enum per faculty user_id: available / busy / do_not_disturb, with a timestamp.
- Scheduler Agent queries availability + existing calendar entries across a candidate faculty set to propose the best overlapping time window and a count of available faculty.

6. API Requirements (High-Level, Representative)

Endpoint (representative)	Purpose
POST /auth/google, POST /auth/signup	Authentication and account creation
POST /rooms, PATCH /rooms/{id}/reschedule	Create/schedule a room; reschedule preserving room_code
POST /rooms/{code}/join	Join a room by code/link (+ password if enabled)
POST /rooms/{id}/recording/start	Start recording (triggers participant notice)
GET /archive?scope=join host	List a user's past meetings (attended, no code required)
POST /archive/unlock	One-time room_code verification for an absentee to unlock an Archive entry
POST /qa/ask	Ask the Q&A Agent a question about a selected meeting_instance_id
POST /qa/escalate	Escalate an unanswered question to a selected host_id
POST /tasks, PATCH /tasks/{id}/progress	Assign a task (single or Select All); update progress
GET /faculty/availability, POST /schedule/suggest	Faculty availability lookup and best-time suggestion
POST /summary/{meeting_instance_id}/approve	Host approves/edits/publishes a draft summary

7. Non-Functional / Engineering Requirements

- Concurrency: the Room Service must generate and validate room codes without collisions under many simultaneous room creations.
- Isolation: query-level enforcement that a host_id's queries can never return another host_id's meeting data (row-level security or equivalent).
- Latency: real-time media path optimized for low-latency audio/video; AI Q&A; responses should return within an interactive timeframe (few seconds), with the app indicating "searching" state while retrieving.
- Resilience: low-bandwidth mode should gracefully degrade to audio-only rather than disconnect.
- Security: recordings and password fields encrypted at rest; optional end-to-end encryption mode for sensitive meetings; domain-restricted room enforcement at join time.

- Auditability: version history retained for host edits to AI-generated summaries.

8. Third-Party Integrations

- Google OAuth for sign-in.
- Managed real-time video SDK (if not building raw WebRTC infra) — e.g. Agora, Daily, or Twilio Video.
- Speech-to-text API (e.g. Whisper) for transcription.
- LLM API for summarization, Q&A;/RAG, and task extraction.
- Push/email notification providers (e.g. SendGrid for email); optional WhatsApp Business API.
- Native OS share sheet for invitation sharing (WhatsApp, Instagram, Facebook, Snapchat, Gmail, SMS, Messenger, Truecaller).